

Day-72-NumPy Indexing and Basic Arithmetic

1. Day number

Day 72

2. Topic name

NumPy indexing, slicing, and arithmetic

3. Connection

Yesterday, in **Day 71**, you created a NumPy array and learned basics such as array, shape, and dtype.

Today, you will learn how to:

- access individual values,
 - select multiple values,
 - perform calculations on every value,
 - find the total,
 - find the average.
-

4. Important topics

Indexing

Indexing means accessing **one value** from an array using its position.

Python starts counting from 0.

```
sales = np.array([100, 200, 300, 400, 500])

sales[0]    # first value
sales[2]    # third value
```

Positions:

Index:	0	1	2	3	4
Value:	100	200	300	400	500

Slicing

Slicing means selecting **a range of values**.

```
sales[1:4]
```

This selects indexes 1, 2, and 3.

The ending index is **not included**.

Addition

You can add a number to every element:

```
sales + 10
```

Multiplication

You can multiply every element:

```
sales * 2
```

sum()

Finds the total of all values:

```
sales.sum()
```

mean()

Finds the average:

```
sales.mean()
```

5. Foundational notes

A NumPy array behaves differently from a normal Python list when doing arithmetic.

Consider:

```
numbers = np.array([10, 20, 30])
```

If you write:

```
numbers + 5
```

NumPy applies + 5 to **each value**.

Conceptually:

```
10 + 5 → 15  
20 + 5 → 25  
30 + 5 → 35
```

Result:

```
[15 25 35]
```

You do not need to manually process every value with a loop for this simple operation.

6. Element-wise operations explained simply

Element-wise means:

| NumPy performs the operation separately on each element of the array.

For example:

```
prices = np.array([10, 20, 30])  
  
prices * 2
```

Think of NumPy doing:

```
10 × 2  
20 × 2  
30 × 2
```

Giving:

```
[20 40 60]
```

Similarly:

```
prices + 5
```

means:

```
10 + 5  
20 + 5  
30 + 5
```

You do not need to study NumPy's detailed broadcasting rules yet. For now, just remember that adding or multiplying an array by **one number** applies that operation to every element.

7. Easy example

Suppose you have daily visitor counts:

```
visitors = np.array([20, 25, 30, 35, 40])
```

Access the first value:

```
visitors[0]
```

Select the first three values:

```
visitors[0:3]
```

Increase every value by 5:

```
new_visitors = visitors + 5
```

Calculate statistics:

```
visitors.sum()  
visitors.mean()
```

Notice how NumPy lets us work with the entire collection without manually changing every value.

8. Problem statement

Create a NumPy array containing these five sales values:

```
100, 150, 200, 250, 300
```

Your program should:

1. Create the NumPy array.
2. Print the complete array.
3. Print the first sales value using indexing.
4. Select a few sales values using slicing.
5. Increase **every sales value by 10**.
6. Print the updated array.
7. Calculate the total of the updated sales.
8. Calculate the average of the updated sales.

Do not modify the problem using advanced NumPy features.

9. Concepts used

You will practice:

```
NumPy  
np.array()  
indexing  
slicing  
element-wise addition  
sum()  
mean()
```

```
variables  
print()
```

The most important new idea is:

```
one array → operation → every element changes
```

10. Thought process

Before writing code, think through the problem in small steps.

First, you need NumPy because the data will be stored in a NumPy array.

Then create the five sales values.

Next, use an index such as:

```
[0]
```

to access one value.

Use a slice such as:

```
[start:end]
```

to retrieve several values.

After that, create an updated array by adding 10 to the sales array.

Finally, use NumPy's built-in operations to calculate:

```
total → sum()  
average → mean()
```

A useful mental model is:

```
Create data  
↓  
Access data  
↓  
Select data  
↓  
Perform arithmetic  
↓  
Calculate summary values
```

11. Beginner-friendly pseudocode

```
START

import NumPy

create an array containing five sales values

print the complete array

get the first sales value using indexing
print it

select some sales values using slicing
print them

add 10 to every sales value
store the result in another variable

print the updated sales

calculate the total using sum()
calculate the average using mean()

print the total
print the average

END
```

12. Suggested solving approach: NumPy approach

Try to solve the arithmetic using NumPy directly rather than writing a loop.

For example, think:

```
updated_sales = sales + ?
```

instead of:

```
loop through every sales value
add something manually
```

Similarly, use NumPy's:

```
.sum()
.mean()
```

rather than manually calculating the total and average.

This is one of the main reasons NumPy is useful for numerical data.

13. Easy edge cases

Empty array

An array can contain no elements:

```
empty = np.array([])
```

Its sum is straightforward, but calculating its mean does not produce a normal meaningful average because there are no values to average.

For beginner programs, you can first check whether the array contains values before calculating the mean.

Zeros

For example:

```
sales = np.array([0, 0, 0, 0, 0])
```

The total would be:

```
0
```

and the average would also be:

```
0
```

Zeros themselves are perfectly valid values.

14. Expected output

For the problem data:

```
100, 150, 200, 250, 300
```

and an increase of 10, your output should look conceptually similar to:

```
Sales: [100 150 200 250 300]
```

```
First sale:
```

```
100
```

```
Selected sales:
```

```
[some selected values]
```

Updated sales:

```
[110 160 210 260 310]
```

Total:

```
1050
```

Average:

```
210.0
```

Your exact labels can be different.

15. Hint only

Start with:

```
import numpy as np
```

Create the array with:

```
np.array(...)
```

Remember:

```
first element → index 0  
several elements → [start:end]  
increase all values → array + number  
total → .sum()  
average → .mean()
```

Try solving the complete program yourself without using a loop.

Quick check before moving to Day 73

You should now understand the difference between:

```
sales[0]
```

which retrieves **one element**, and:

```
sales[0:3]
```

which retrieves **multiple elements**.

You should also understand why:

```
sales + 10
```

can update every element in one simple NumPy operation.